

Concurrent and Distributed Systems

Lab 5 – Safety and Liveness

Materials

More about Java concurrency classes: **More on Java Concurrency.pptx**

A useful summary (cheat sheet) for Java concurrency: **Understanding_the_Core_Concurrency_Concepts.pdf**

Archive with source code: **cds.lab5.zip**. This contains the following packages:

- `bridge.unsafe` (single lane bridge problem - cars crashing on the bridge)
- `bridge.safe` (single lane bridge problem - safe and fair implementation)
- `cds.lab5` (`RaceConditionAtomicInteger`, `ParallelSort`, `BlockingQueueExample`)
- `balls` (bouncing balls)

Lab session

Activity 1 - Examples

Try the programs (included in the archive) presented in the lecture and additional slides:

- Compare the implementations in the packages **bridge.unsafe** and **bridge.safe** with respect to safety and fairness.
- In the package **cds.lab5**, check out the use of `AtomicInteger` in **RaceConditionAtomicInteger**, the parallel execution in **ParallelSort**, the producer-consumer problem using an `ArrayBlockingQueue` in **BlockingQueueExample**.

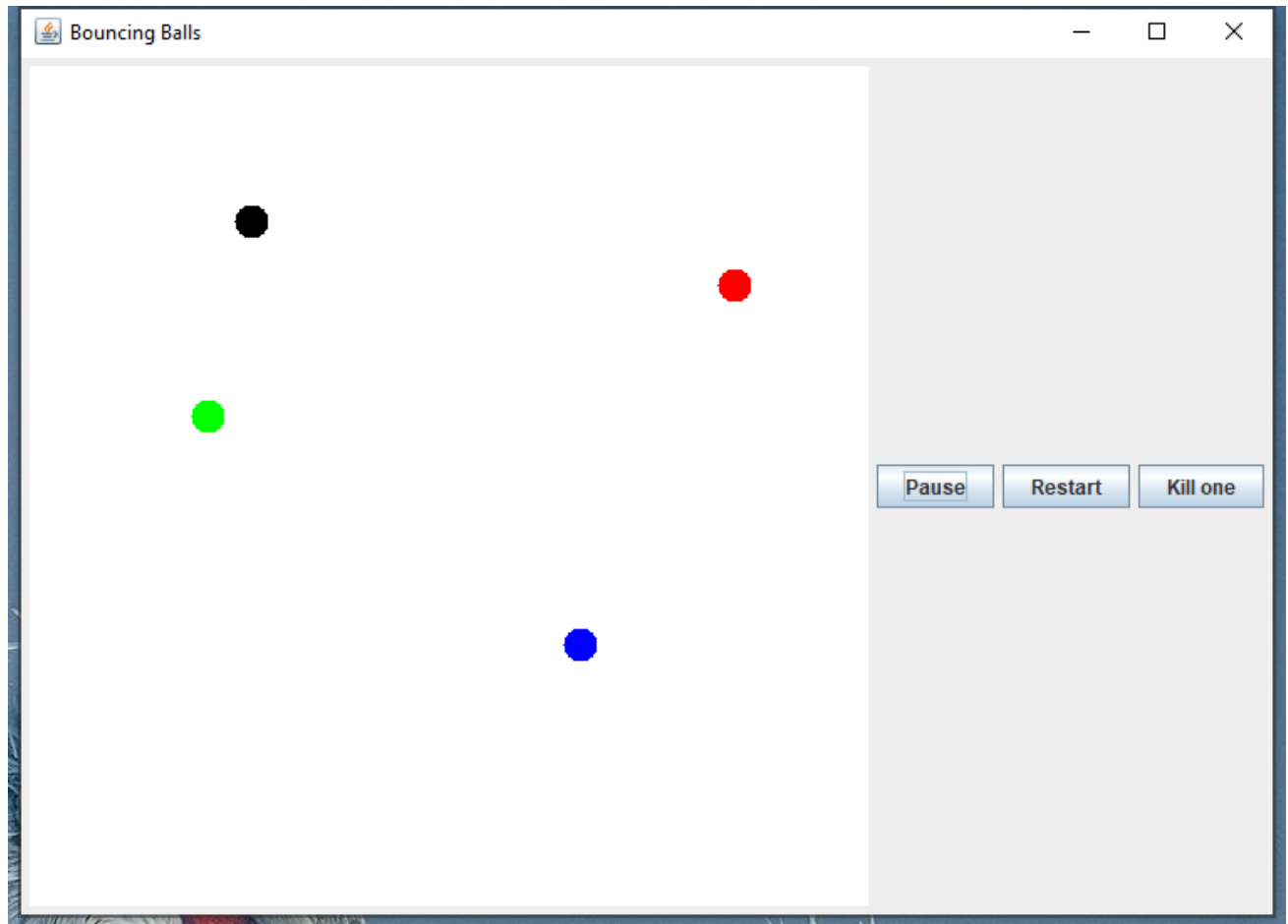
Activity 2 - Bouncing balls

The package **balls** contains 3 classes:

- **Ball.java**: an instance of this class is a ball that moves in a `BallsPanel`. The `Ball` class is a subclass of `Thread`, which runs an infinite loop where the ball repeatedly calls `doMove()` - this updates its position, calls `panel.repaint()` to repaint and sleeps for a short while (30 ms). The ball also has the ability to draw itself, given a graphics context. Furthermore, upon creation (i.e. in its constructor), the ball registers itself with the panel it appears in by calling `addBall()`.
- **BallsMainFrame.java**: this class contains the main method, which creates a Swing window with some buttons and a panel with a couple of balls. The buttons don't have any action implemented yet - this should be provided by you.
- **BallsPanel.java**: this is a subclass of `JPanel`, which can contain several balls, stored in an `ArrayList`. The panel draws itself in `paintComponent()` by asking all its contained balls to draw themselves.

Modify the code to implement the actions of the buttons:

- "Pause" - should pause all the balls movement (freeze them at the current location).
- "Restart" - should restart the balls movement.
- "Kill one" - should kill a random ball, that is, terminate the `run()` method of the thread (without using a deprecated method to stop the thread!) and remove the ball from the panel.



Resources

- J. Fernandez Gonzalez (2017), **Java 9 Concurrency Cookbook** (2nd edition), Packt Publishing. You can access the online version through the library (log in with your university account), link [here](#). The GitHub repository providing the complete Java classes for each book chapter example is here: <https://github.com/PacktPublishing/Java-9-Concurrency-Cookbook-Second-Edition>
- KVS Prasad (Chalmers University of Technology & Gothenburg University). Concurrent Programming module http://www.cse.chalmers.se/edu/year/2016/course/TDA383_LP1/java-exercises.html